

The ARC Disaster-Response Game

A Formal Specification of Game Mechanics

(Unity engine + gym interface boundary)

CORA / ARC_Game

June 24, 2026

Abstract

This document formalizes the mechanics of the ARC disaster-response management game as implemented in the Unity engine and exposed to decision-making policies (LLMs and non-learning heuristics) through the TCP gym interface. It defines the temporal model, the action space and its enumeration, the task lifecycle, building and worker mechanics, client-stay and casework dynamics, transportation and delivery, the flooding/weather stochastic layer, and the budget/termination economy. All numeric constants reflect the *headless gym build*, which is the configuration policies actually run against; where a value is decided in the Python action layer rather than in Unity, this is stated explicitly. The Python-side reward function is intentionally out of scope and treated as a separate artifact; here we document only the raw quantities Unity reports.

Contents

1	Overview and Notation	2
2	Temporal Model and the Step Sequence	3
2.1	Discrete time	3
2.2	One round of wall-clock simulation	3
2.3	Ordering within a step	3
2.4	Initial state (reset)	4
3	Action Space	4
3.1	Enumeration and the round trip	4
3.2	Construction	5
3.3	Workers: hire and train	5
3.4	Worker assignment	5
3.5	Resource transfer	6
3.6	Deconstruction	6
4	Tasks	6
4.1	Data model	6
4.2	Types and tags	6
4.3	Generation	6
4.4	Choice feasibility (gating)	7
4.5	Resolution and fulfillment accounting	7

5	Buildings and Capacities	7
5.1	Inventory of building types	7
5.2	Status automaton	8
5.3	Food production and consumption	8
6	Workers	9
6.1	Pools and workforce value	9
6.2	Assignment	9
6.3	Person-round accounting	9
7	Client Stay and Casework	9
8	Transportation and Delivery	9
8.1	Delivery configuration on a choice	9
8.2	Source/destination resolution	10
8.3	Occupancy update and fulfillment	10
9	Flooding, Weather, and Non-Determinism	10
9.1	Weather	10
9.2	Flood dynamics	10
9.3	Stochasticity and reproducibility	11
9.4	What the policy observes	11
10	Economy and Termination	11
10.1	Budget	11
10.2	Recurring cost: motel	11
10.3	Satisfaction and termination	11
A	Numeric Constants Reference	11

1 Overview and Notation

The player (“Director”) manages a disaster-relief operation over a fixed horizon. Each round the player may build and deconstruct facilities, hire and train workers, assign workers to facilities, transfer resources, and respond to *tasks* (events) by selecting among offered choices. Decisions consume a shared *budget* and influence *satisfaction*; floods and weather perturb the map and block delivery routes. The objective (scored externally in Python) rewards meeting population needs (food, lodging, casework/return-home) while spending efficiently.

Agent–environment loop. A policy interacts with the game through the gym wrapper (`arc_game_gym_env_tcp.py`) over TCP to a headless Unity process (`GymServerManager.cs`). Each environment *step* consists of (i) executing zero or more queued actions, then (ii) advancing the simulation by exactly one round, then (iii) returning the updated game state. The detailed ordering is given in §2.

Notation. Table 1 lists recurring symbols.

Symbol	Meaning
$d \in \{1, \dots, 8\}$	current day
$s \in \{0, 1, 2, 3\}$	time segment within a day (9:00, 12:00, 15:00, 18:00)
$r = 4(d - 1) + s + 1$	global round index (1–32); 0-based in the payload
B	current budget (dollars)
Σ	current satisfaction, $\Sigma \in [0, 100]$
W_T, W_U	free trained / untrained worker counts
$\omega(\cdot)$	workforce value: $\omega(\text{trained}) = 2$, $\omega(\text{untrained}) = 1$
ρ_b	required workforce of building b (always 4 for constructables)
$\text{cap}_b^{\text{pop}}, \text{cap}_b^{\text{food}}$	population / food capacity of building b
n_b	current occupancy (population) of building b

Table 1: Notation.

Source-of-truth convention. Numeric constants are defined canonically in the Unity build; we annotate these (**U**). The gym action layer transmits a per-action **cost** that Unity charges verbatim, so the charge mechanism lives in Python while the *value* must mirror the Unity canonical constant — the two are required to stay in sync. A few quantities exist *only* in the Python action layer (the resource-transfer quantity menus and the per-action hire-bundle cap); these are annotated (**P**). Quantities overridable by external config are annotated (**c**fg).

2 Temporal Model and the Step Sequence

2.1 Discrete time

Time is partitioned into *days* and *rounds*. There are 4 rounds (time segments) per day and at most 8 days, giving a maximum horizon of

$$r_{\max} = 4 \times 8 = 32 \text{ rounds.}$$

The natural episode terminus is the end of Day 8, Round 4 (`GlobalClock.cs`), and the benchmark harness plays the full 32-round horizon. The gym wrapper exposes a `max_episode_steps` cap (one step = one round) as a safety bound; it is set at or above 32 so it does not truncate a normal episode — the episode ends at the natural Day 8/Round 4 terminus (or early on satisfaction collapse, §10). Round indexing in the serialized payload is 0-based ($r-1$).

2.2 One round of wall-clock simulation

In headless mode a round is advanced by a single `advance_time` request, which runs one *simulation coroutine*. The coroutine spins for a fixed game-time budget using a fixed capture timestep:

$$\Delta t = 0.3 \text{ game-s/frame}, \quad T_{\text{round}} = 10 \text{ game-s}, \quad N_{\text{frames}} = \lceil T_{\text{round}} / \Delta t \rceil = 34.$$

Thus roughly 10.2 game-seconds elapse per round. This matters because several in-engine timers are expressed in game-seconds (construction 5 s, deconstruction 3 s); since each is shorter than one round’s budget, they *complete within the same round* in which they start (§5).

2.3 Ordering within a step

The precise order of effects in a single environment step is given in Algorithm 1. The key consequence for policies: **queued actions are applied before the simulation runs**, and **within-**

round resolutions (deliveries arriving, tasks expiring, food produced/consumed, costs billed) become observable only in the state returned at the end of the step.

Algorithm 1 Environment step (one round)

```

1: Input: list of action indices  $A$  (possibly empty), choice selections
2: for each action  $a \in A$  do                                     ▷ all execute before time advances
3:   EXECUTEACTION( $a$ )                                           ▷ build/deconstruct, hire/train/assign, queue transfer
4:   deduct cost( $a$ ) from  $B$ ; mutate worker pools / sites immediately
5: end for
6: for each pending choice selection do                         ▷ select_task_choice
7:   apply choice impacts (satisfaction now; budget now if cost, scheduled if income)
8:   trigger immediate delivery, or enqueue a vehicle delivery (§8)
9: end for
10: if  $s \geq 4$  then                                             ▷ day boundary reached on the previous round
11:   PROCEEDTONEXTDAY :  $d += 1$ ;  $s \leftarrow 0$ 
12:   fire ONDAYCHANGED: charge motel occupancy cost; clear food waste
13: end if
14: RUNSIMULATIONCOROUTINE( $T_{\text{round}}$ )                             ▷ ~10s of physics
   vehicles move/deliver; construction & deconstruction coroutines finish;
   kitchens produce food; shelters/community consume food; task timers tick
15: ENDSIMULATION :
   REWARDMETRICSTRACKER.ONROUNDEDED (snapshot worker person-rounds)
    $s += 1$ ; fire ONROUNDEDED (apply matured delayed-budget items)
   check end-of-day / end-of-game
16: return serialized GameStatePayload; re-enumerate valid actions

```

2.4 Initial state (reset)

A fresh episode initializes to: $d = 1$, $s = 0$; budget $B = 10000$ and satisfaction $\Sigma = 50$ (both overridable by config, §10); worker pool of 5 trained + 5 untrained, all immediately free; the prebuilt map (Community at full occupancy, Motel empty, abandoned construction sites); and no player-built facilities. Tasks are generated on demand each round (§4).

3 Action Space

3.1 Enumeration and the round trip

Before each decision the action layer (`action_enumerator.py`) enumerates *all currently valid actions* from the latest game state, producing an indexed list. A policy selects one or more indices; the gym serializes each selected action to JSON and sends it to Unity, where `ActionExecutor` deserializes it into a `GameAction` and applies the corresponding effect, charging exactly the `cost` field carried in the message. After the round advances, the action list is regenerated, so *indices are not stable across rounds*.

There are six action families. Five are enumerated mutating actions; the sixth, `select_task_choice`, answers a task (§4) and is issued through a dedicated path rather than the mutating-action list.

Family	Cost	Availability predicate (enumeration gate)
construction	\$1000	site available $\wedge B \geq 1000$
worker (hire/train)	see §3.3	budget/pool gates (no daily cap)
worker_assignment	\$0	building $\in \{\text{NeedWorker}, \text{InUse}\} \wedge$ unmet workforce
resource_transfer	\$0	a vehicle available $\wedge$ source stock $\wedge$ dest space
deconstruction	\$0	target is a player-built building
select_task_choice	impacts of the choice	task active $\wedge$ choice feasible

Table 2: Action families.

3.2 Construction

For each available abandoned site and each constructable type $\tau \in \{\text{Kitchen}, \text{Shelter}, \text{CaseworkSite}\}$ with $B \geq 1000$, one action `build_ $\tau$ _{site}` is enumerated. Executing it deducts \$1000, marks the site occupied, and creates the building in status UNDERCONSTRUCTION. The building completes (transitions to NEEDWORKER) within the same round (§2).

3.3 Workers: hire and train

Worker actions are *quantity-bundled*: for an affordable quantity range the enumerator emits one action per quantity q , with cost linear in q . There is **no per-day hiring ceiling** — hiring is bounded only by the budget. A per-action bundle cap $M_{\text{hire}} = 5$ (and $M_{\text{train}} = 10$) (**P**) keeps the enumerated list bounded; a policy may still hire every round and issue multiple hire actions per step, so cumulative hiring is unlimited.

$$\text{hire untrained: cost} = 100q, \quad q \in \{1, \dots, \min(\lfloor B/100 \rfloor, M_{\text{hire}})\} \quad (1)$$

$$\text{hire trained: cost} = 300q, \quad q \in \{1, \dots, \min(\lfloor B/300 \rfloor, M_{\text{hire}})\} \quad (2)$$

$$\text{train untrained: cost} = 500q, \quad q \in \{1, \dots, \min(W_U, \lfloor B/500 \rfloor, M_{\text{train}})\} \quad (3)$$

In the headless build, hiring and training are *instantaneous*: a hired or newly trained worker is immediately available (status FREE); there is no arrival delay and no multi-round training period. Training converts one untrained worker ($\omega = 1$) into one trained worker ($\omega = 2$) in place. Note the resulting economics: training a worker (\$500) costs *more* than hiring a trained worker outright (\$300), so under these constants direct trained-hire dominates training as a way to add trained workforce.¹

3.4 Worker assignment

For each building needing workforce, the enumerator emits assign actions in both worker classes for quantities up to the shortfall. Let the shortfall be $\delta_b = \rho_b - \text{assigned}(b)$. The executor assigns *trained-first, then untrained* to satisfy the requested headcount, drawing from the free pools, and updates building status when $\sum \omega(\cdot) \geq \rho_b$ (§6).

¹The interactive (human) build routes hiring/training through separate request/training systems with arrival delays; those paths are not exercised by the gym and are out of scope here. The worker costs above are the canonical Unity values (\$100/\$300/\$500), which the gym action layer mirrors and charges.

3.5 Resource transfer

For each ordered (source, destination) facility pair and each fixed quantity in a preset menu, a transfer is enumerated when source stock and destination free space both admit it and a vehicle is available:

food packs: $q \in \{10, 25, 50, 100\}$, population: $q \in \{5, 10, 20\}$.

The transfer costs nothing to issue; it creates a vehicle delivery task that resolves over the round (§8).

3.6 Deconstruction

For each player-built building, `deconstruct_{name}` is enumerated. Executing it releases all assigned workers back to the free pool and begins deconstruction, which completes within the same round, freeing the site for future construction.

4 Tasks

4.1 Data model

A `GameTask` bundles: a type and optional tag, an affected facility, a countdown `roundsRemaining`, a list of *impacts* (signed deltas on Satisfaction, Budget, FoodPacks, Clients, Workforce, ...), and a list of *choices* (`AgentChoice`). Population/lodging tasks also carry a `demandQuantity` (people needing service) and an accumulating `deliveredQuantity`. A choice carries its own impacts and an optional *delivery configuration* (§8).

4.2 Types and tags

The *type* sets the response window and penalty semantics; the *tag* routes the task into reward accounting.

Type	Default window	Expiry semantics
Emergency	1 round	on expiry: incomplete; impacts (penalties) applied
Demand	2 rounds	on expiry: incomplete; impacts (penalties) applied
Advisory	3 rounds	may be ignored with no penalty
Alert	2 rounds	handled globally; not an interactive task-center item
Other	—	non-displaying; auto-dismissed
		Tag
		Reward routing
		Food
		food resolved/fulfilled
		Lodging
		lodging resolved/fulfilled
		BackToHome
		casework/return-home
		None
		not scored

Table 3: Task types (left) and tags (right).

4.3 Generation

At the start of each round the task system evaluates a database of task templates. Each template is gated by one or more *triggers* combined with AND or OR logic, evaluated per eligible facility (or

once globally for global tasks). Duplicate suppression prevents re-spawning an identical (facility, title) task that is already active. The trigger taxonomy:

- **Deterministic:** Round, Day (specific/interval/range/from), Resource threshold (empty/full/</>), Budget, Satisfaction, Workforce, FacilityStatus, Weather, FloodTile count, FloodedFacility (proximity).
- **Stochastic:** Probability trigger — fires when a fresh uniform draw $u \sim U[0, 1)$ satisfies $u < p$, evaluated independently per facility. Community-request generation similarly uses a per-community Bernoulli(0.5).

Flood-driven tasks (road blockage, vehicle repair) are event-driven, emitted when the flood layer expands onto a route or a vehicle is caught (§9). Because probability triggers, weather, and floods draw from an unseeded RNG, task generation is *stochastic across episodes* (§9.3).

4.4 Choice feasibility (gating)

A choice with no delivery is always feasible. A delivery choice is feasibility- checked before being offered as selectable:

- **Food delivery:** immediate (airlift) variants are always feasible; deferred variants require kitchen stock and an available vehicle.
- **Population relocation:** requires destination capacity — shelter space if routing to shelters, motel space if routing to motels.
- **Return-home / casework:** routing to a CaseworkSite is treated as processing-home and is *not* gated on shelter/motel space, so the casework option is never hidden when it is the intended response.

An infeasible choice is shown with a reason but cannot be selected.

4.5 Resolution and fulfillment accounting

Selecting a choice applies its impacts immediately (satisfaction; budget cost immediately, budget income possibly delayed — §10) and either completes the task at once (no delivery), or sets it IN-PROGRESS pending delivery completion. A task ends as COMPLETED (fully serviced), INCOMPLETE (expired with penalties), or ignored (Advisory).

For Food/Lodging-tagged tasks the metrics tracker records, at resolution,

$$\text{resolved} += \begin{cases} D & D > 0 \\ 1 & D = 0 \end{cases}, \quad \text{fulfilled} += \begin{cases} \min(\text{delivered}, D) & D > 0 \\ \mathbb{1}[\text{serviced}] & D = 0, \end{cases}$$

where $D = \text{demandQuantity}$. For a Lodging task, D is set at creation to the *maximum* delivery quantity across the task’s choices. A delivery that arrives *after* the task has closed (a “late delivery”) still increments the fulfilled count (capped at resolved), crediting service that physically occurred.

5 Buildings and Capacities

5.1 Inventory of building types

Table 4 lists every building. Constructable buildings cost \$1000, require workforce 4, and are built on abandoned sites; prebuilt buildings exist from the start and need no workforce.

Building	Kind	Pop cap	Food cap	Workforce	Cost	Role
Kitchen	constructable	—	20	4	\$1000	produces 10 food/round
Shelter	constructable	10	10	4	\$1000	houses people; consumes food
CaseworkSite	constructable	40	—	4	\$1000	processes people home
Community	prebuilt	40	40	—	—	origin population (starts full); consumes food
Motel	prebuilt	300	—	—	—	overflow lodging; \$200/person/day

Table 4: Building inventory and capacities (headless build).

5.2 Status automaton

Every building moves through a status lifecycle (Fig. 1). A constructable starts UNDERCONSTRUCTION; on completion it becomes NEEDWORKER; once enough workforce is assigned ($\sum \omega \geq \rho_b = 4$) it becomes INUSE and is *operational* (only operational buildings produce/consume and can source/receive deliveries). Losing workforce drops it back to NEEDWORKER. Deconstruction releases workers and removes the building.

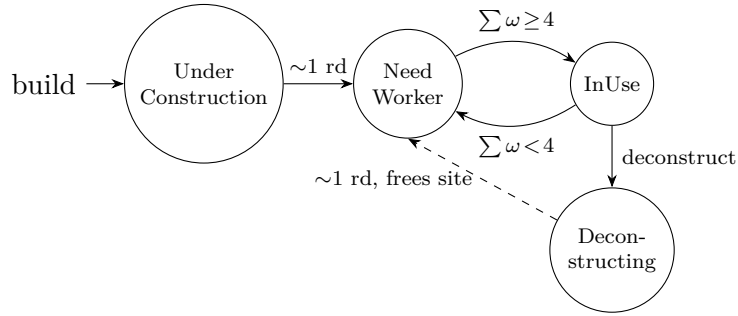


Figure 1: Building status lifecycle. Construction and deconstruction timers (5s and 3s of game time) each complete within a single round.

5.3 Food production and consumption

A Kitchen, when operational, produces a fixed amount per round up to its food capacity:

$$\text{food}_{\text{Kitchen}}(r+1) = \min(\text{cap}_{\text{Kitchen}}^{\text{food}}, \text{food}_{\text{Kitchen}}(r) + 10).$$

Buildings with population-based consumption enabled (Shelter, Community) consume one food pack per resident per $\kappa = 4$ rounds. Letting $c_b(r)$ be a per-building consumption phase counter,

$$c_b += 1; \quad \text{if } c_b \equiv 0 \pmod{\kappa}: \quad \text{food}_b -= n_b \quad (\text{capped at available stock}).$$

Motel and CaseworkSite do not consume food. Unconsumed food may be cleared at the day boundary (food-waste rule). Workers do not consume food in the current configuration.

6 Workers

6.1 Pools and workforce value

Workers are *trained* ($\omega = 2$) or *untrained* ($\omega = 1$) and occupy one of the statuses FREE, WORKING, or (untrained only) TRAINING. The available workforce is

$$\Omega_{\text{free}} = 2W_T + W_U.$$

The episode begins with 5 trained and 5 untrained workers ($\Omega = 2 \cdot 5 + 5 = 15$), all free.

6.2 Assignment

`TryAssignWorkersToBuilding( $b, q$ )` fills building b toward its required headcount by drawing *trained workers first, then untrained*, setting each chosen worker to WORKING bound to b , until either q workers are placed or the free pools are exhausted. A building activates (`NEEDWORKER` $\rightarrow$ `INUSE`) once its assigned workforce value reaches $\rho_b = 4$; e.g. two trained workers (2×2) or four untrained (4×1) suffice.

6.3 Person-round accounting

At each round end the metrics tracker accumulates worker allocation as person-rounds, $\sum_r(\text{working, training, idle})$, forming the basis for the externally computed labor-cost/utilization terms. These are reported raw; no scoring is applied in Unity.

7 Client Stay and Casework

People relocated into lodging (Shelter or Motel) are tracked as *client groups* with an arrival round. The relevant round measure is $r = 4(d-1) + s$. A group’s residence duration drives the return-home (casework) dynamics:

- **Casework request threshold** = 8 rounds. Once a group has resided ≥ 8 rounds, a casework (return-home, `BACKTOHOME`) task is generated for it. Servicing it promptly grants satisfaction; delaying it costs satisfaction.
- **Overstay threshold** = 8 rounds: groups past threshold are flagged overstaying and tracked until processed.

Casework is *processed* by relocating the group to a `CaseworkSite`: a population delivery whose destination is a `CaseworkSite` removes those clients from lodging and credits “processed-home” in the metrics. Both Shelters and Motels generate casework demand, coupling lodging occupancy to the casework workload: people housed earlier mature into casework demand later in the episode.

8 Transportation and Delivery

8.1 Delivery configuration on a choice

A choice that moves resources carries a delivery spec: a cargo type (Population or FoodPacks); a quantity rule (a fixed amount, a percentage of available, or “all”); a *source* and *destination* resolution mode (the requesting facility, a specific named building, a building/prebuilt *type* to search, or auto-find); and nearest-first preference flags. Two execution modes exist:

Immediate An instantaneous transfer: no vehicle, no route, no travel time. It succeeds to the extent destination capacity allows; any unplaceable remainder is returned to the source. Population relocation that moves zero people leaves the task open (no false completion).

Vehicle-queued A `DeliveryTask` is created and assigned to a vehicle that physically loads at the source, travels a flood-aware path, and unloads at the destination over the course of the round. Creation is blocked up front if no flood-free route exists; en-route flooding can still interrupt a vehicle (§9).

8.2 Source/destination resolution

For type- or auto-resolved endpoints the game enumerates matching buildings, filters to those that can actually source the cargo (have stock) or receive it (have free space), and — when nearest-first is set — chooses the one closest (Euclidean) to the requesting facility, excluding the source from being chosen as its own destination. Effective free space subtracts already-in-flight inbound reservations so concurrent vehicles do not over-commit a destination.

8.3 Occupancy update and fulfillment

On unload, population is removed from the source’s occupancy and added to the destination’s, respecting capacity; the same *population-delivery hook* that updates occupancy also updates client-stay tracking (§7) — registering arrivals at lodging or processing clients home at a `CaseworkSite`. The delivered quantity is credited to the parent task’s `deliveredQuantity`; completion of all linked deliveries closes the task (or, if it already closed, is credited as a late delivery).

9 Flooding, Weather, and Non-Determinism

9.1 Weather

Each day a weather condition is drawn (e.g. Clear/Rain/Storm) via a probability-weighted selection on the day boundary. Weather sets flood dynamics parameters (spawn bonus, spread multiplier, shrinkage) and determines whether new flooding can appear: rain above a minimum intensity is required to spawn flood tiles.

9.2 Flood dynamics

Flooding is represented as a set of flooded grid tiles. Each round, subject to weather, the flood layer:

- **spawns** at river tiles with probability $p_{\text{spawn}} = p_0 + \beta \cdot \text{rainIntensity}$ (when rain is sufficient);
- **spreads** to neighboring tiles with a base probability scaled by a weather multiplier and a terrain factor (land easy, terrain-blocked hard);
- performs occasional **random jumps** (teleporting expansion) with a small probability, to a random nearby tile;
- **shrinks** per-tile with a probability that rises sharply when rain has stopped (drying out).

Flooded tiles over a road block delivery routes; flooded tiles near a facility mark it flood-affected. A vehicle that enters a flooded tile mid-route is damaged and stopped, which fails its delivery (applying the task’s delivery-failure penalty) and emits road-blockage / vehicle-repair tasks.

9.3 Stochasticity and reproducibility

The game’s randomness is concentrated in three subsystems: *weather selection* (daily), *flood spawn/spread/jump/shrink* (per round), and *probability-based task triggers* (per facility per round). All draw from Unity’s global RNG, which is **not seeded**; consequently episodes are *not reproducible* — identical action sequences yield different weather and flood trajectories. Vehicle damage, by contrast, is deterministic given the flood layout (it is a geometric collision check, not a random draw).

9.4 What the policy observes

The serialized environment conditions expose a coarse view: the current weather *label* (not the numeric intensity), a binary flooding flag, the count of blocked roads, the count of damaged vehicles, and an aggregate flood state (active flag, affected-road count). Spatial flood layout and the specific blocked routes are not itemized in the payload.

10 Economy and Termination

10.1 Budget

The budget starts at \$10000 (config-overridable). Spending is categorized (Food, Lodging, Worker, Casework, Other) and accumulated per category for the external cost terms. Costs are charged immediately when incurred; *income* granted by a task choice may be *delayed*: a choice can schedule a positive budget impact ΔB to mature after k rounds, applied at the round-end event once its countdown reaches zero. Costs are never delayed.

10.2 Recurring cost: motel

The single recurring charge is the motel occupancy cost, billed on each day boundary:

$$\text{cost}_{\text{motel}}(d) = 200 \times n_{\text{Motel}} \quad (\text{category: Lodging}).$$

This makes the motel free to enter but expensive to leave occupied — the dominant lever in lodging cost-efficiency.

10.3 Satisfaction and termination

Satisfaction $\Sigma \in [0, 100]$ is moved by task impacts and casework timeliness. An episode **terminates** when $\Sigma \leq 0$, and is otherwise **truncated** at the end of Day 8 Round 4 (or earlier, at the gym’s `max_episode_steps`). The starting satisfaction is 50 (config-overridable). The only quantities read from external config in the headless build are the initial budget and initial satisfaction; all other constants in this document are fixed in the build.

A Numeric Constants Reference

Quantity	Value	Source	Notes
Rounds per day	4	U	segments 9:00/12:00/15:00/18:00
Max days	8	U	natural terminus Day 8 Round 4

Quantity	Value	Source	Notes
Max rounds	32	U	4×8
Round game-time budget	10 s	U	~ 34 frames @ 0.3 s
Capture timestep	0.3 s	U	headless fixed delta
Initial budget	\$10000	U/cfg	config-overridable
Initial satisfaction	50	U/cfg	config-overridable; range [0, 100]
Initial trained workers	5	U	all free at reset
Initial untrained workers	5	U	all free at reset
Construction cost	\$1000	U	all constructable types; charged via action layer
Construction time	5 s (≈ 1 round)	U	completes same round
Deconstruction time	3 s (≈ 1 round)	U	completes same round
Required workforce	4	U	every constructable building
Hire untrained	\$100 / worker	U	instantaneous, free on hire
Hire trained	\$300 / worker	U	instantaneous, free on hire
Train untrained $\rightarrow$ trained	\$500 / worker	U	instantaneous (headless); $>$ trained-hire
Per-action hire bundle cap	5	P	no per-day ceiling; budget-limited only
Per-action training cap	10	P	enumeration bound only
Trained workforce value	2	U	
Untrained workforce value	1	U	
Kitchen food capacity	20	U	
Kitchen production	10 food / round	U	when operational
Shelter capacity	10 pop, 10 food	U	consumes food
CaseworkSite capacity	40 pop	U	no consumption
Community capacity	40 pop, 40 food	U	starts full; consumes food
Motel capacity	300 pop	U	no food consumption
Motel cost	\$200 / person / day	U	billed at day boundary
Food consumption rate	1 / person / 4 rounds	U	Shelter, Community
Transfer menu (food)	{10,25,50,100}	P	per (src,dst) pair
Transfer menu (population)	{5,10,20}	P	per (src,dst) pair
Casework request threshold	8 rounds	U	residence duration
Overstay threshold	8 rounds	U	
Emergency window	1 round	U	
Demand window	2 rounds	U	
Advisory window	3 rounds	U	ignorable, no penalty

Table 5: Authoritative constants for the headless gym build.
U = value fixed canonically in the Unity build (action costs are charged via the gym action layer, which mirrors the Unity value); **P** = value defined only in the Python action layer; **cfg** = overridable by external config.